

LuaTools Skill HTTP API（中文）

[简体中文](#) ▼

本文档说明 LuaTools 内置 Skill 服务提供的 HTTP / WebSocket 接口。

(关于LuaTools的基础介绍, 请移步于此[点击了解LuaTools](#);))

- 基础地址: `http://127.0.0.1:38380`
- 除 WebSocket 外, 请求体与响应体均为 JSON。

1. 接口总览

- `GET /skill/health`: 健康检查
- `GET /skill/capabilities`: 获取可执行动作列表
- `GET /skill/flash/status`: 获取刷机状态 (只读快捷接口)
- `POST /skill/action`: 执行动作
- `GET ws://127.0.0.1:38380/skill/events`: 订阅运行时事件流 (WebSocket)

2. 健康检查

`GET /skill/health`

用于检查服务是否在线。

响应示例:

```
{
  "ok": true,
  "version": "1.0.0",
  "service": "luatools-skill",
  "time": 1760000000.123
}
```

字段说明:

- `ok`: 固定为 `true`。
- `version`: LuaTools 版本号。
- `service`: 固定为 `luatools-skill`。

- `time`：服务端时间戳（秒，浮点数）。

测试：

```
curl http://127.0.0.1:38380/skill/health
```

3. 查询能力

GET /skill/capabilities

返回当前服务支持的 action 名称列表。

响应示例：

```
{
  "ok": true,
  "actions": [
    "flash.firmware",
    "project.flash",
    "flash.status",
    "project.combine",
    "project.export",
    "project.import",
    "project.list",
    "project.details",
    "project.create",
    "project.update",
    "project.delete",
    "project.syntax_check",
    "combine.firmware",
    "log.list",
    "log.tail",
    "serial.list",
    "serial.select",
    "serial.open",
    "serial.close",
    "serial.status",
    "serial.mode",
    "serial.baudrate",
    "trace.start",
    "trace.stop",
    "trace.status",
    "device.reboot"
  ],
  "confirm_required": []
}
```

字段说明：

- `actions`：可调用动作名列表。

- `confirm_required`：当前版本固定为空数组 `[]`。

测试：

```
curl http://127.0.0.1:38380/skill/capabilities
```

3.1 刷机状态（只读快捷接口）

GET /skill/flash/status

用途：直接获取刷机状态（等价于调用 `POST /skill/action + flash.status`），便于轮询。

响应格式：

```
{
  "ok": true,
  "result": {
    "run_id": 12,
    "active": true,
    "status": "running",
    "overall_progress": 63,
    "current_step": "write_ap",
    "steps": []
  }
}
```

测试：

```
curl http://127.0.0.1:38380/skill/flash/status
```

4. 执行动作

POST /skill/action

统一执行入口。

请求格式：

```
{
  "action": "action.name",
  "params": {}
}
```

通用字段说明：

- `action` (必填, `string`): 动作名。
- `params` (选填, `object`): 动作参数; 省略时等同 `{}`。

成功响应格式:

```
{
  "ok": true,
  "result": {}
}
```

失败响应格式:

```
{
  "ok": false,
  "error": "action_failed"
}
```

说明: `error` 会是服务内部错误码 (见“错误说明”章节)。

4.1 `flash.firmware`

用途: 烧录固件包。

参数说明:

- `firmware_path` (必填, `string`): 固件文件完整路径; 必须存在。
- `usb_boot` (选填, `boolean`): 是否启用 USB Boot; 默认 `false`。
- `clear_kv` (选填, `boolean`): 是否清除 KV; 默认 `false`。
- `clear_fs` (选填, `boolean`): 是否清除文件系统; 默认 `false`。

请求示例:

```
{
  "action": "flash.firmware",
  "params": {
    "firmware_path": "D:/firmware.binpkg",
    "usb_boot": false,
    "clear_kv": false,
    "clear_fs": false
  }
}
```

4.2 `project.flash`

用途: 烧录项目。

参数说明:

- `project_name` (必填, `string`): 项目名, 必须是现有项目。
- `mode` (选填, `string`): 烧录模式; 可选范围: `script` / `all`, 默认 `script`。

请求示例:

```
{
  "action": "project.flash",
  "params": {
    "project_name": "demo_project",
    "mode": "script"
  }
}
```

4.3 `project.combine`

用途: 组合项目产物 (工厂包/升级包)。

参数说明:

- `project_name` (必填, `string`): 项目名, 必须是现有项目。
- `output_dir` (选填, `string`): 输出目录路径; 不传时由 UI/默认流程决定。

请求示例:

```
{
  "action": "project.combine",
  "params": {
    "project_name": "demo_project",
    "output_dir": "D:/output"
  }
}
```

4.4 `project.export`

用途: 导出项目为 `.luatos` 文件。

参数说明:

- `project_name` (必填, `string`): 项目名, 必须是现有项目。
- `output_dir` (选填, `string`): 导出目录; 未提供时默认导出到当前工作目录下 `projexp/`。

请求示例:

```
{
  "action": "project.export",
  "params": {
    "project_name": "demo_project",
  }
}
```

```
    "output_dir": "D:/projexp"
  }
}
```

4.5 project.import

用途：导入 `.luatos` 项目包。

参数说明：

- `file_path` (必填, `string`)：待导入文件路径；必须存在且扩展名为 `.luatos`。
- `project_name` (选填, `string`)：导入后的项目名；不传时由文件名自动推断。
- `overwrite` (选填, `boolean`)：项目已存在时是否覆盖；可选范围：`true` / `false`，默认 `false`。

请求示例：

```
{
  "action": "project.import",
  "params": {
    "file_path": "D:/pkg/demo_project.luatos",
    "project_name": "demo_project",
    "overwrite": false
  }
}
```

4.6 project.list

用途：列出项目。

参数说明：无参数，传 `{}` 即可。

请求示例：

```
{
  "action": "project.list",
  "params": {}
}
```

4.7 project.details

用途：获取项目详细信息（配置及文件列表）。

参数说明：

- `project_name` (必填, `string`)：项目名，必须是现有项目。

请求示例：

```
{
  "action": "project.details",
  "params": {
    "project_name": "demo_project"
  }
}
```

返回说明：

- `project_name`：项目名。
- `info`：项目配置信息（`[info]` 节所有字段）。
- `file_sections`：文件目录及文件绝对路径列表。
- `ini_path`：项目配置文件完整路径。

返回示例：

```
{
  "ok": true,
  "result": {
    "project_name": "demo_project",
    "info": {
      "core_path": "D:/firmware.soc",
      "type": ".soc",
      "output_path": "D:/output",
      "lib": "",
      "demo": ""
    },
    "file_sections": {
      "D:/github/luatools_py3/userprojs/demo_project":
      ["D:/github/luatools_py3/userprojs/demo_project/main.lua"]
    },
    "ini_path": "D:/github/luatools_py3/project/demo_project.ini"
  }
}
```

4.8 `project.create`

用途：创建新项目。

参数说明：

- `project_name`（必填，`string`）：新项目名称，不能与现有项目重名。
- `core_path`（选填，`string`）：底层 core 文件路径（.soc/.binpkg 等）。
- `script_path`（选填，`string`）：脚本目录或脚本文件路径。
- `resource_path`（选填，`string`）：资源文件目录或资源文件路径。

- `output_path` (选填, `string`): 量产文件输出目录。

注意: 若提供的路径不存在, 将返回错误 `directory_not_found`。

请求示例:

```
{
  "action": "project.create",
  "params": {
    "project_name": "new_project",
    "core_path": "D:/firmware.soc",
    "script_path": "D:/scripts",
    "resource_path": "D:/resources",
    "output_path": "D:/output"
  }
}
```

返回说明:

- `project_name`: 创建的项目名。

4.9 `project.update`

用途: 更新项目配置。

参数说明:

- `project_name` (必填, `string`): 项目名, 必须是现有项目。
- `updates` (必填, `object`): 要更新的字段及值, 支持以下字段:
- `core_path`: 底层 core 路径。
- `output_path`: 输出目录。
- `output_suffix_enable`: 是否启用输出文件后缀 (`true` / `false`)。
- `output_suffix`: 输出文件后缀字符串。
- `lib`: 库名称。
- `demo`: demo 名称。
- `print_mode`: 打印模式 (整数)。
- `add_core`: 升级包是否包含 core (`true` / `false`)。
- `only_code`: 仅生成脚本 (`true` / `false`)。
- `only_luac_code`: 仅生成 luac 脚本 (`true` / `false`)。
- `default_lib`: 使用默认库 (`true` / `false`)。
- `code_enable`: 启用代码加密 (`true` / `false`)。
- `code`: 代码加密密码。

- `luac_debug`：保留 `luac` 调试信息（`true` / `false`）。
- `lua_nocheck`：忽略脚本依赖检查（`true` / `false`）。
- `add_file`：添加文件到项目（文件完整路径）。
- `remove_dir`：删除指定目录下的所有文件（目录路径）。
- `clear_files`：清除所有文件条目（值为 `true` 时执行）。
- 其它任意 `[info]` 节字段（通用设置）。

请求示例：

```
{
  "action": "project.update",
  "params": {
    "project_name": "demo_project",
    "updates": {
      "core_path": "D:/new_firmware.soc",
      "output_path": "D:/new_output",
      "add_file": "D:/scripts/extra.lua"
    }
  }
}
```

返回说明：

- `updated`：是否更新成功（固定 `true`）。

4.10 `project.delete`

用途：删除项目（仅删除 `.ini` 配置文件，不删除脚本/资源目录）。

参数说明：

- `project_name`（必填，`string`）：项目名，必须是现有项目。

请求示例：

```
{
  "action": "project.delete",
  "params": {
    "project_name": "demo_project"
  }
}
```

返回说明：

- `deleted`：是否删除成功（固定 `true`）。

4.11 `project.syntax_check`

用途：对项目中的 Lua 脚本进行语法检查。

参数说明：

- `project_name` (必填, `string`)：项目名，必须是现有项目。
- `is_float` (选填, `boolean`)：是否使用浮点数版本 luac；默认 `false`。

请求示例：

```
{
  "action": "project.syntax_check",
  "params": {
    "project_name": "demo_project",
    "is_float": false
  }
}
```

返回说明：

- `errors`：语法错误列表；若为空数组表示检查通过。
- `message` (可选)：当无 Lua 文件时返回 `no_lua_files`。

返回示例（成功）：

```
{
  "ok": true,
  "result": {
    "errors": []
  }
}
```

返回示例（失败）：

```
{
  "ok": true,
  "result": {
    "errors": ["main.lua:5: syntax error near 'then'"]
  }
}
```

4.12 `combine.firmware`

用途：合并底包和脚本/二进制文件。

参数说明：

- `core_path` (必填, `string`)：底包路径；必须存在，后缀仅支持 `.soc` 或 `.binpkg`。
- `bin_path` (必填, `string`)：待合并 `bin` 文件路径；必须存在。

- `address` (必填, `string`): 十六进制地址字符串; 格式范围: `0x` 开头, 后续仅 `0-9a-fA-F`, 例如 `0x220000`。

请求示例:

```
{
  "action": "combine.firmware",
  "params": {
    "core_path": "D:/firmware.soc",
    "bin_path": "D:/script.bin",
    "address": "0x220000"
  }
}
```

4.13 `log.list`

用途: 列出日志目录文件。

参数说明: 无参数, 传 `{}` 即可。

请求示例:

```
{
  "action": "log.list",
  "params": {}
}
```

返回说明:

- `log_dir`: 日志目录绝对路径。
- `files`: 日志文件名列表 (按名称降序, 最多返回前 200 条)。

4.14 `log.tail`

用途: 读取日志文件末尾内容。

参数说明:

- `file_path` (必填, `string`): 日志文件完整路径; 必须存在。
- `max_lines` (选填, `integer`): 返回行数; 可选范围: 整数且 ≥ 1 , 默认 200。

请求示例:

```
{
  "action": "log.tail",
  "params": {
    "file_path": "D:/github/luatools_py3/log/xxx.log",
    "max_lines": 200
  }
}
```

```
}  
}
```

4.15 flash.status

用途：获取当前（或最近一次）刷机状态，包含整体进度和子过程状态。

参数说明：无参数，传 {} 即可。

请求示例：

```
{  
  "action": "flash.status",  
  "params": {}  
}
```

返回字段说明：

- run_id：刷机任务序号（每次新刷机自增）。
- active：是否仍在刷机中。
- action：任务来源（如 flash.firmware、project.flash）。
- status：整体状态：idle / running / success / failed。
- overall_progress：整体进度百分比（0-100）。
- current_step：当前子过程 ID（例如 write_ap、write_cp、erase_fs、erase_fskv）。
- steps：子过程列表，每项包含：
 - id：子过程标识。
 - name：子过程名称。
 - status：子过程状态：pending / running / completed / failed。
 - progress：子过程进度百分比（0-100）。
 - last_info：最近一条下载状态描述文本。
 - last_error：失败时的错误文本。
 - started_at / updated_at / finished_at：状态时间戳（秒）。

返回示例：

```
{  
  "ok": true,  
  "result": {  
    "run_id": 12,  
    "active": true,  
    "action": "flash.firmware",  
    "params": {
```

```

        "firmware_path": "D:/firmware.binnpk",
        "usb_boot": false,
        "clear_kv": true,
        "clear_fs": true
    },
    "status": "running",
    "overall_progress": 63,
    "current_step": "write_ap",
    "steps": [
        { "id": "prepare", "name": "固件准备", "status": "completed", "progress": 100 },
        { "id": "connect", "name": "连接设备", "status": "completed", "progress": 100 },
        { "id": "erase_fskv", "name": "清除FSKV", "status": "completed", "progress": 100 },
        { "id": "erase_fs", "name": "清除FS", "status": "completed", "progress": 100 },
        { "id": "write_ap", "name": "刷写AP分区", "status": "running", "progress": 63 },
        { "id": "write_cp", "name": "刷写CP分区", "status": "pending", "progress": 0 }
    ],
    "last_info": "正在下载, 请勿断电或者断开连接线",
    "last_error": null,
    "started_at": 1760000000.111,
    "updated_at": 1760000018.222,
    "finished_at": null
}

```

4.16 serial.list

用途：获取可用串口列表。

参数说明：无参数，传 {} 即可。

请求示例：

```

{
  "action": "serial.list",
  "params": {}
}

```

返回说明：

- ports：串口列表，每项包含：
- name：串口名称（如 COM3）。
- description：串口描述。
- hwid：硬件ID。

返回示例：

```
{
  "ok": true,
  "result": {
    "ports": [
      { "name": "COM3", "description": "USB Serial Port", "hwid": "USB VID:PID=1234:5678" },
      { "name": "COM4", "description": "COM4", "hwid": "BTHENUM" }
    ]
  }
}
```

4.17 serial.select

用途：选择要使用的串口。

参数说明：

- `port_name`（必填，`string`）：串口名称（如 `COM3`）。

请求示例：

```
{
  "action": "serial.select",
  "params": {
    "port_name": "COM3"
  }
}
```

返回说明：

- `selected`：是否选择成功（固定 `true`）。
- `port_name`：选中的串口名称。

4.18 serial.open

用途：打开当前选中的串口。

参数说明：无参数，传 `{}` 即可。

请求示例：

```
{
  "action": "serial.open",
  "params": {}
}
```

返回说明：

- `opened`：是否打开成功（固定 `true`）。

- `port_name`：打开的串口名称。

错误码：

- `no_ports_available`：无可用的串口。
- `port_already_open`：串口已经打开。
- `no_port_selected`：未选择串口。

4.19 `serial.close`

用途：关闭当前打开的串口。

参数说明：无参数，传 `{}` 即可。

请求示例：

```
{
  "action": "serial.close",
  "params": {}
}
```

返回说明：

- `closed`：是否关闭成功（固定 `true`）。

错误码：

- `port_not_open`：串口未打开。

4.20 `serial.status`

用途：获取当前串口状态。

参数说明：无参数，传 `{}` 即可。

请求示例：

```
{
  "action": "serial.status",
  "params": {}
}
```

返回说明：

- `current_port`：当前选中的串口名称。
- `is_open`：串口是否已打开。
- `baudrate`：当前波特率。

- `mode`：当前模式（`usb` / `4g` / `host` / `unknown`）。
- `trace_on`：日志打印是否开启。

返回示例：

```
{
  "ok": true,
  "result": {
    "current_port": "COM3",
    "is_open": true,
    "baudrate": 115200,
    "mode": "4g",
    "trace_on": true
  }
}
```

4.21 `serial.mode`

用途：设置串口模式（USB/4G/HOST）。

参数说明：

- `mode`（必填，`string`）：模式名称；可选范围：`usb` / `4g` / `host`。

请求示例：

```
{
  "action": "serial.mode",
  "params": {
    "mode": "4g"
  }
}
```

返回说明：

- `mode_set`：是否设置成功（固定 `true`）。
- `mode`：设置的模式。

错误码：

- `missing_mode`：缺少 `mode` 参数。
- `invalid_mode`：无效的 `mode` 值。
- `close_port_first`：串口已打开，需先关闭再切换模式。

4.22 `serial.baudrate`

用途：设置串口波特率。

参数说明：

- `baudrate` (必填, `integer`)：波特率值；可选范围：1200 - 6000000 (常见值：9600, 19200, 38400, 57600, 115200, 230400, 460800, 921600 等)。

请求示例：

```
{
  "action": "serial.baudrate",
  "params": {
    "baudrate": 115200
  }
}
```

返回说明：

- `baudrate_set`：是否设置成功 (固定 `true`)。
- `baudrate`：设置的波特率值。

错误码：

- `missing_baudrate`：缺少 `baudrate` 参数。
- `invalid_baudrate`：无效的波特率值。
- `close_port_first`：串口已打开，需先关闭再设置波特率。

4.23 `trace.start`

用途：开始日志打印 (打开串口日志输出到界面)。

参数说明：无参数，传 `{}` 即可。

请求示例：

```
{
  "action": "trace.start",
  "params": {}
}
```

返回说明：

- `started`：是否启动成功 (固定 `true`)。

错误码：

- `trace_already_running`：日志打印已经在运行。

4.24 `trace.stop`

用途：停止日志打印。

参数说明：无参数，传 `{}` 即可。

请求示例：

```
{
  "action": "trace.stop",
  "params": {}
}
```

返回说明：

- `stopped`：是否停止成功（固定 `true`）。

错误码：

- `trace_not_running`：日志打印未在运行。

4.25 `trace.status`

用途：获取日志打印状态。

参数说明：无参数，传 `{}` 即可。

请求示例：

```
{
  "action": "trace.status",
  "params": {}
}
```

返回说明：

- `is_running`：日志打印是否正在运行。
- `com_open`：串口是否已打开。
- `current_port`：当前串口名称。

返回示例：

```
{
  "ok": true,
  "result": {
    "is_running": true,
    "com_open": true,
    "current_port": "COM3"
  }
}
```

4.26 device.reboot

用途：重启已连接的设备。

参数说明：无参数，传 `{}` 即可。

请求示例：

```
{
  "action": "device.reboot",
  "params": {}
}
```

返回说明：

- `rebooted`：是否发送重启命令成功（固定 `true`）。

错误码：

- `reboot_failed`：重启失败（底层设备错误）。

5. 事件流（WebSocket）

`ws://127.0.0.1:38380/skill/events`

连接后服务端会持续推送 JSON 事件。

常见 `type`：

- `hello`：连接建立后首条欢迎消息。
- `core_event`：LuaTools 内核事件转发。
- `action_started`：动作开始执行。
- `action_failed`：动作执行失败。
- `action_finished`：动作执行结束。

事件示例：

```
{
  "type": "action_started",
  "action": "project.flash",
  "params": {
    "project_name": "demo_project",
    "mode": "script"
  },
  "error": null,
}
```

```
"ts": 1760000000.123
}
```

字段说明：

- `action`：动作名。
- `params`：动作参数回显。
- `error`：仅失败事件有值，其余通常为 `null`。
- `ts`：服务端时间戳（秒，浮点数）。

测试：

```
wscat -c ws://127.0.0.1:38380/skill/events
```

6. 错误说明

6.1 常见错误码

接口可能返回（包括但不限于）以下错误码：

- `invalid_json`
- `missing_action`
- `unknown_action`
- `missing_firmware_path`
- `firmware_not_found`
- `missing_project_name`
- `project_not_found`
- `missing_file_path`
- `file_not_found`
- `missing_parameters`
- `unsupported_core_type`
- `project_exists`
- `directory_not_found`
- `invalid_project_config`
- `file_not_found`
- `invalid_update_field`
- `invalid_extension`

- `invalid_project_file`
- `ui_not_ready`
- `missing_port_name`
- `port_not_found`
- `no_ports_available`
- `port_already_open`
- `no_port_selected`
- `port_not_open`
- `missing_mode`
- `invalid_mode`
- `close_port_first`
- `missing_baudrate`
- `invalid_baudrate`
- `baudrate_set_failed`
- `trace_already_running`
- `trace_not_running`
- `reboot_failed`

6.2 HTTP 状态码

- `200`：请求成功，或动作返回结果。
- `400`：请求体非法（例如 JSON 格式错误、缺少 `action`）。
- `500`：动作执行阶段异常。

7. 兼容性说明

本文档仅覆盖当前 LuaTools 内置 Skill HTTP/WebSocket API 的实现行为。

8. 快速调用示例（Windows PowerShell）

以下示例默认服务地址为：`http://127.0.0.1:38380`。

8.1 健康检查

```
curl.exe http://127.0.0.1:38380/skill/health
```

8.2 查询能力

```
curl.exe http://127.0.0.1:38380/skill/capabilities
```

8.3 执行动作通用模板

```
$body = @{  
    action = "project.list"  
    params = @{}  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action `  
-H "Content-Type: application/json" `  
-d $body
```

8.4 flash.firmware

```
$body = @{  
    action = "flash.firmware"  
    params = @{  
        firmware_path = "D:/firmware.binpkg"  
        usb_boot = $false  
        clear_kv = $false  
        clear_fs = $false  
    }  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.5 project.flash

```
$body = @{  
    action = "project.flash"  
    params = @{  
        project_name = "demo_project"  
        mode = "script"  
    }  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.6 project.combine

```
$body = @{
    action = "project.combine"
    params = @{
        project_name = "demo_project"
        output_dir = "D:/output"
    }
} | ConvertTo-Json -Depth 10

curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type: application/json" -d $body
```

8.7 project.export

```
$body = @{
    action = "project.export"
    params = @{
        project_name = "demo_project"
        output_dir = "D:/projexp"
    }
} | ConvertTo-Json -Depth 10

curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type: application/json" -d $body
```

8.8 project.import

```
$body = @{
    action = "project.import"
    params = @{
        file_path = "D:/pkg/demo_project.luatos"
        project_name = "demo_project"
        overwrite = $false
    }
} | ConvertTo-Json -Depth 10

curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type: application/json" -d $body
```

8.9 project.list

```
$body = @{
    action = "project.list"
    params = @{}
} | ConvertTo-Json -Depth 10

curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type: application/json" -d $body
```

8.10 project.details

```
$body = @{  
    action = "project.details"  
    params = @{  
        project_name = "demo_project"  
    }  
}  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.11 project.create

```
$body = @{  
    action = "project.create"  
    params = @{  
        project_name = "new_project"  
        core_path = "D:/firmware.soc"  
        script_path = "D:/scripts"  
        resource_path = "D:/resources"  
        output_path = "D:/output"  
    }  
}  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.12 project.update

```
$body = @{  
    action = "project.update"  
    params = @{  
        project_name = "demo_project"  
        updates = @{  
            core_path = "D:/new_firmware.soc"  
            output_path = "D:/new_output"  
        }  
    }  
}  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.13 project.delete

```
$body = @{  
    action = "project.delete"  
    params = @{  
        project_name = "demo_project"  
    }  
}
```

```
}  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.14 project.syntax_check

```
$body = @{  
    action = "project.syntax_check"  
    params = @{  
        project_name = "demo_project"  
        is_float = $false  
    }  
}  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.15 combine.firmware

```
$body = @{  
    action = "combine.firmware"  
    params = @{  
        core_path = "D:/firmware.soc"  
        bin_path = "D:/script.bin"  
        address = "0x220000"  
    }  
}  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.16 log.list

```
$body = @{  
    action = "log.list"  
    params = @{}  
}  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.17 log.tail

```
$body = @{  
    action = "log.tail"  
    params = @{}  
}
```

```
file_path = "D:/github/luatools_py3/log/xxx.log"
max_lines = 200
}
} | ConvertTo-Json -Depth 10

curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:
application/json" -d $body
```

8.18 订阅事件流

```
wscat -c ws://127.0.0.1:38380/skill/events
```

8.19 serial.list

```
$body = @{
    action = "serial.list"
    params = @{}
} | ConvertTo-Json -Depth 10

curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:
application/json" -d $body
```

8.20 serial.select

```
$body = @{
    action = "serial.select"
    params = @{
        port_name = "COM3"
    }
} | ConvertTo-Json -Depth 10

curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:
application/json" -d $body
```

8.21 serial.open

```
$body = @{
    action = "serial.open"
    params = @{}
} | ConvertTo-Json -Depth 10

curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:
application/json" -d $body
```

8.22 serial.close

```
$body = @{  
    action = "serial.close"  
    params = @{}  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.23 serial.status

```
$body = @{  
    action = "serial.status"  
    params = @{}  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.24 serial.mode

```
$body = @{  
    action = "serial.mode"  
    params = @{  
        mode = "4g"  
    }  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.25 serial.baudrate

```
$body = @{  
    action = "serial.baudrate"  
    params = @{  
        baudrate = 115200  
    }  
} | ConvertTo-Json -Depth 10  
  
curl.exe -X POST http://127.0.0.1:38380/skill/action -H "Content-Type:  
application/json" -d $body
```

8.26 trace.start

```
$body = @{  
    action = "trace.start"  
    params = @{}  
} | ConvertTo-Json -Depth 10
```

